

7/22/23, 2:42 AM

CodioProjectFileEditFindViewToolsEducationHelpNode versionProject Index (static)Configure...

Img_Invertin... x

1# WRITE YOUR CODE HERE

100% (1:1)

Codio - Transformation

Guide x

Collapse

Inverting Images

Inverting an image involves reversing its color values, creating a “negative” of the original image. This can be useful for creating interesting visual effects, emphasizing specific features of an image, or generating additional variations of a generated image. We will explore image inversion using the Python Imaging Library (PIL) and the DALL-E 2 API.

```
import os
import openai
import secret
from PIL import Image, ImageOps
from io import BytesIO
import requests

openai.api_key = secret.api_key

#Saving our base image
base_image=Image.open('base_img.jpg')
```

Color Inversion

Color inversion involves reversing the RGB values of each pixel in the image. This can create a striking effect that highlights different aspects of the image compared to the original.

```
# Invert the colors of the base image
inverted_image = ImageOps.invert(base_image)

# Save the inverted image to a file
inverted_image.save('inverted.jpg')
```

TRY IT!

Command was successfully executed.

Click here to refresh your inverted image

Grayscale Inversion

Grayscale inversion involves first converting the image to grayscale and then reversing its color values. This creates a negative effect while preserving the tonal relationships in the image.

```
# Convert the base image to grayscale
grayscale_image = ImageOps.grayscale(base_image)

# Invert the grayscale image
inverted_grayscale_image = ImageOps.invert(grayscale_image)

# Save the inverted grayscale image to a file
inverted_grayscale_image.save('inverted_grayscale.jpg')
```

TRY IT!

Command was successfully executed.

Click here to refresh your grayscale inverted image

Inversion with Custom Color Channels

You can also create custom effects by inverting only specific color channels of the image. For example, you can invert the red and green channels while preserving the blue channel.

```
# Split the color channels of the base image
r, g, b = base_image.split()

# Invert the red and green channels
inverted_r = ImageOps.invert(r)
inverted_g = ImageOps.invert(g)

# Combine the inverted red and green channels with the original blue channel
inverted_custom_channels_image = Image.merge("RGB", (inverted_r, inverted_g, b))

# Save the custom channel inverted image to a file
inverted_custom_channels_image.save('inverted_custom_channels.jpg')
```

TRY IT!

Click here to refresh your inverted custom channels image

By applying various inversion techniques to images generated by the DALL-E 2 API, you can create unique and visually engaging images, adding variety and interest to your projects.

To make a negative of an image image using the Python Imaging Library (PIL), you can use the `ImageOps.invert()` function.

☐ rotate

☒ invert

☐ contrast

☐ grayscale

The `ImageOps.invert()` function is used to invert the colors of an image in PIL. The function subtracts the value of each pixel from the maximum value possible for that channel. This operation effectively flips the colors, making dark areas light and light areas dark. For example, a white pixel (RGB value of (255, 255, 255)) would become black (RGB value of (0, 0, 0)), and a black pixel (RGB value of (0, 0, 0)) would become white (RGB value of (255, 255, 255)). The `ImageOps.invert()` function can be a useful tool for creating negative versions of images or for creating artistic effects by inverting the color scheme.

Check It!

https://codio.com/sandipan-dey/transformation1/1